

MOSS-TTS Integration for vLLM-Omni

Andrew Chung
Department of Computer Science
Columbia University
New York City, United States
ac5905

Yutao Mao
Department of Computer Science
Columbia University
New York City, United States
ym3019

Yuting Liu
Department of Computer Science
Columbia University
New York City, United States
yl5961

Xingru Lu
Department of Computer Science
Columbia University
New York City, United States
xl3602

Abstract—vLLM has become a widely used inference serving system for large language models because of its efficient scheduling, KV-cache management, and batching support [1]. vLLM-Omni extends the same serving direction to multimodal generation, including speech and audio generation [2]. However, text-to-speech (TTS) support in vLLM-Omni is still early, and the MOSS-TTS family had not been integrated into the vLLM ecosystem in this repository before this project.

This project integrates two MOSS-TTS variants, MOSS-TTS-Local and MOSS-TTS-Delay, into the vLLM-Omni two-stage pipeline [3]. We additionally wire async-chunk streaming paths so Stage 1 can begin decoding partial code chunks before Stage 0 finishes the full utterance. For the Delay model, the integration adds incremental async de-delay, vectorized sync de-delay, packed audio embeddings and heads, vectorized active-codebook sampling, dynamic async chunk sizing, and reduced CPU/runtime transfer overhead in the async connector path. In the current benchmark tables, the strongest measured speedup is observed for MOSS-TTS-Delay synchronous throughput at batch size 256, where vLLM-Omni reaches 69.956 in the reported throughput metric compared with 25.006 for the HuggingFace baseline. For time to first audio, MOSS-TTS-Local async chunking reduces first chunk latency from 21.860 s in the native HuggingFace baseline to 0.557 s in vLLM-Omni.

Index Terms—vLLM, vLLM-Omni, MOSS-TTS, MOSS-TTS-Local, MOSS-TTS-Delay, text-to-speech, async chunking, throughput, real-time factor, time to first audio

I. INTRODUCTION

A. Background

High-quality text-to-speech (TTS) models are increasingly important for interactive assistants, accessibility tools, and real-time speech interfaces. These workloads have serving requirements that differ from offline audio synthesis: users care not only about total generation time, but also about time to first audio (TTFA), steady-state throughput, and the ability to stream partial audio while the model continues generating the rest of an utterance.

vLLM-Omni represents multimodal generation as a staged pipeline [2]. A stage can consume the user prompt, consume a previous stage’s output, or produce final modality output. For TTS, this naturally maps to a two-stage design: Stage 0 maps text to discrete acoustic codes, and Stage 1

maps those codes to waveform audio. The repository uses `stage_configs` YAML files to bind model classes, schedulers, workers, memory budgets, and inter-stage processors. In our MOSS-TTS configs, Stage 0 uses the autoregressive worker and `OmniARScheduler`; Stage 1 uses the generation worker and `OmniGenerationScheduler`. Stage 0 emits latent outputs containing `code_predictor_codes`; Stage 1 receives flattened code tokens and returns audio.

B. Problem Statement

Our goal is to integrate MOSS-TTS into vLLM-Omni and conducting a profiling-driven performance study of throughput and latency on a single GPU. However, there were two challenges. First, MOSS-TTS does not follow a standard HuggingFace model interface, so vLLM-Omni needs custom model adapters and weight-loading logic. Second, the MOSS audio tokenizer decoder is not a standard text transformer; it is a separate 1.6B-parameter CAT codec model that consumes 32-layer residual vector quantization (RVQ) audio codes and produces waveform audio [4].

These challenges are more significant in a streaming serving setting because the system must coordinate autoregressive code generation, inter-stage transfer, and codec decoding while preserving low TTFA and high throughput.

C. Scope

We implement this integration for both MOSS-TTS-Local and MOSS-TTS-Delay. MOSS-TTS-Local uses a global Qwen3-1.7B backbone together with a local transformer [5]. During audio generation, the global backbone provides cross-frame context, while the local transformer predicts one full 32-codebook RVQ frame per decode step. The code path tracks whether each request is in audio mode and only invokes the local audio predictor for decode positions that are actually emitting audio.

MOSS-TTS-Delay uses an 8B delay-pattern autoregressive model. Instead of emitting a complete frame-major RVQ row directly, it emits delayed rows where quantizer channels are shifted relative to each other. Before the shared CAT codec

can decode the output, the integration must invert this delay pattern and drop rows that still contain the audio pad sentinel.

The current branch, `moos-tts-delay`, contains the Delay integration. The Local implementation and optimization work are in the `moos-tts-local-optimization` branch.

II. LITERATURE REVIEW

A. Efficient LLM and Multimodal Serving

Efficient LLM serving systems such as vLLM optimize autoregressive inference through improved scheduling, batching, and KV-cache management [1], but are primarily designed for single-stage text generation. vLLM-Omni extends this to multimodal pipelines via a stage-graph abstraction, decomposing any-to-any models into independently served stages with per-stage batching and scheduling [2]. This directly motivates our MOSS-TTS integration: Stage 0 performs autoregressive audio code generation, and Stage 1 decodes RVQ codes into waveforms, with streaming connectors enabling asynchronous chunked decoding for lower time to first audio.

B. MOSS-TTS

MOSS-TTS is an open-source speech generation foundation model built on discrete audio tokens, autoregressive modeling, and large-scale multilingual pretraining [3]. It builds upon MOSS-Audio-Tokenizer that employs causal Transformers to tokenize 24 kHz audio down to 12.5 fps using 32 layers of variable bitrate RVQ [4]. Two complementary models, both employing the same tokenizer and pretraining procedure, are released, which are Delay Pattern model (MOSS-TTS) that involves only one Transformer backbone with multiple prediction heads and a delay schedule aware of RVQ. The other model is called Local-Transformer model that includes a frame-local autoregressive component for improved speaker preservation and faster initial speech generation.

III. METHODOLOGY

A. Data

For our experiments, we utilized the wikitext dataset for our RTF and FCL analysis, as well as fixed sentences for the throughput benchmark. For the wikitext benchmark, we utilized 20 sentences of varying lengths from 10-60 words. We attempted longer paragraph variants, but would hit OOM errors. The specific fixed sentence we utilized was "The weather is so nice today and the birds are singing in the trees."

B. Model

1) *Unified Model Wrapper:* The integration exposes MOSS-TTS through `MossTTSForConditionalGeneration`. This wrapper dispatches to one of three internal stages according to the `model_stage` field in the YAML configuration:

- `ar_stage`: MOSS-TTS-Local autoregressive code generation.
- `delay_ar_stage`: MOSS-TTS-Delay autoregressive code generation.

- `decoder`: shared CAT codec waveform decoding.

This keeps the vLLM-Omni model registry interface stable while allowing the two TTS variants to use different Stage 0 implementations and the same Stage 1 decoder.

2) *MOSS-TTS-Local Stage 0*: The Local Stage 0 model reconstructs the original MOSS-TTS-Local generation logic inside vLLM-Omni. It builds a vLLM Qwen3 backbone from the language configuration, creates 33 channels of embeddings and heads, and instantiates the MOSS local transformer from the model's remote code. Channel 0 corresponds to text tokens; channels 1–32 correspond to audio RVQ codebooks. The audio codebook size is 1024, and the integration treats code 1024 as the pad sentinel.

For each active audio decode step, the model projects the global hidden state into the local-transformer hidden space, runs the local transformer channel by channel, samples audio codes for channels 1–32, and stores the resulting row as `code_predictor_codes` with shape $[B, 1, 32, 1]$. The implementation also maintains a per-request finite-state machine for entering and exiting audio mode. This is necessary because MOSS-TTS interleaves text-control tokens with audio generation tokens; after the model enters audio mode, the text channel is gated so the vLLM sampler can choose between continuing audio generation and ending audio generation.

The Local integration also includes a streaming configuration. In that mode, Stage 0 uses asynchronous scheduling and a custom next-stage processor, `llm2decoder_async_chunk`, to flush completed frame-major RVQ rows to Stage 1 through a shared-memory connector. Because Local emits a full [32]-codebook frame per decode step, the first audio chunk only requires `codec_chunk_frames` generated frames. The optimized Local async config sets `codec_chunk_frames=3`, prioritizing low TTFA over peak high-batch throughput.

3) *MOSS-TTS-Delay Stage 0*: The Delay Stage 0 model integrates the 8B delay-pattern generator. It maintains per-request delay state and samples audio rows from model logits while masking the audio pad code out of the valid sampling set. The model emits the same `code_predictor_codes` tensor shape as the Local path, but the semantic meaning is different: each output row follows the delay schedule rather than the frame-major layout expected by the CAT decoder.

The integration handles this mismatch in the stage input processor. For sync mode, `llm2decoder_delay` collects all delayed rows, applies the inverse delay transform

$$\text{frame}[t, q] = \text{delayed}[t + q, q],$$

filters incomplete rows containing the pad sentinel, flattens the restored $[T, 32]$ code matrix row-major, and sends the result to Stage 1.

The Delay Stage 0 path also adopts several optimizations from the native MOSS-TTS implementation style. Audio embeddings are packed into a single `audio_embedding_weight` tensor with shape $[n_{vq}, \text{vocab}, \text{hidden}]$, allowing all active audio-channel embeddings for a decode row to be gathered and summed

in one tensor operation. Audio LM heads are similarly packed into `audio_lm_head_weight`; logits for all 32 codebooks are produced with one batched `einsum` rather than 32 separate head calls. Audio sampling is vectorized over the active request/codebook pairs, so inactive delay slots remain padded while active slots are sampled together. When `repetition_penalty=1.0`, which is the benchmark setting, the implementation does not copy every generated audio row to CPU for repetition history bookkeeping.

4) *Shared CAT Decoder Stage:* Both variants use `MossTTSTDecoderModel` as Stage 1. The decoder loads the separate MOSS-Audio-Tokenizer checkpoint, either from an explicit `audio_tokenizer_path`, the `MOSS_AUDIO_TOKENIZER_PATH` environment variable, or the main model path. The codec exposes a decode API that converts codes shaped as $[32, T]$ into 24 kHz waveform audio.

The Stage 1 implementation parses flattened per-request code streams back into code matrices, pads batches where needed, and decodes them through the CAT codec. For `async-chunk` mode, the decoder can enter the codec’s streaming context and preserve causal state across chunks for the same request. The implementation also supports a batched streaming path when multiple active requests are decoded through one shared codec streaming context.

5) *Inter-Stage Processing:* The Local sync processor, `llm2decoder`, collects the Stage 0 `code_predictor_codes`, normalizes them to $[T, 32]$, and flattens them row-major into $[T \times 32]$. The Delay sync processor performs the same final flattening, but first applies the inverse delay transform described above.

For `async chunking`, the Local path uses `llm2decoder_async_chunk`. Because Local emits complete frame-major rows, the processor can flush each group of `codec_chunk_frames` new frames directly to the decoder. The optimized Local config uses a shared-memory connector and sets `codec_chunk_frames` to 3 for low first-audio latency.

The Delay `async path` uses `llm2decoder_delay_async_chunk`. This path cannot flush immediately after three delay rows because the first complete frame-major row is only recoverable after all 32 quantizer positions have had time to arrive. The processor therefore maintains an incremental inverse-delay ring buffer per request. It places each incoming delayed row into restored-frame slots, drains only complete rows in frame order, and flushes once enough complete rows are pending. The Delay `async config` uses a small `initial_codec_chunk_frames` value for TTFA and a larger steady-state `codec_chunk_frames` value for throughput.

C. Serving Optimizations

This project focuses on inference-time serving optimization rather than model training or fine-tuning. We do not modify the training objective, update model weights, or perform additional

supervised training. Instead, all experiments use fixed pre-trained MOSS-TTS model checkpoints, and the optimization scope is limited to integrating the models into vLLM-Omni and improving their inference execution path. The integrations use three categories of serving optimization.

Stage-level batching. Both models are mapped onto vLLM-Omni’s staged schedulers: Stage 0 runs under `OmniARScheduler`, and Stage 1 runs under `OmniGenerationScheduler`. This lets vLLM-Omni batch requests at the stage boundary instead of treating each full TTS request as a separate end-to-end Python loop. The synchronous path benefits most from this design at large batch sizes because Stage 1 receives full code sequences and can decode them with less streaming overhead.

Async two-stage decoding. Both MOSS-TTS-Local and MOSS-TTS-Delay have `async-chunk` configurations. In `async` mode, Stage 0 does not wait for a full utterance before invoking Stage 1. Instead, a custom next-stage processor emits partial code chunks through a shared-memory connector, and the CAT decoder streams waveform chunks while Stage 0 continues generating later RVQ codes. This is the main mechanism behind the TTFA reductions in Table V.

KV-cache management for MOSS-TTS-Local. Stage 0 of MOSS-TTS-Local contains a global temporal transformer and a local depth transformer. The local transformer was invoked repeatedly during autoregressive code generation in the original implementation, where the local context grows one step at a time. In other words, the local transformer has to recompute attention over the entire accumulated local context at every step, resulting in redundancy. As a result, an explicit KV-cache support was added so each local step processes only the newly appended token while reusing the cached key-value states from prior steps. In profiling, this change reduced the CUDA time of the Stage 0 local-transformer section by approximately 8%.

Delay-specific compute and transfer reductions. The Delay branch adds incremental `async de-delay`, vectorized `sync de-delay`, packed audio embeddings, packed audio heads, vectorized active audio sampling, conditional repetition history tracking, hidden-state copy suppression for the MOSS `async connector path`, scheduler-output suppression for unnecessary per-step pooling payloads, and connector cleanup for per-request Delay `async state`. These changes reduce work in the Stage 0 hot path and reduce Python/runtime overhead around `async chunk transfer`.

D. Profiling Tools

We mainly utilized WandB to track our runs, and attempted the use of `pytorch profiler` and the integration of this profiler based in vLLM-Omni. However, we were not able to extract many useful insights, as we find that the types of information extracted from these profiling methodologies are different, especially due to the two stage architecture that we had implemented, vs. the combined architecture of native HuggingFace Models.

TABLE I
VERIFICATION OF DELAY INTEGRATION OPTIMIZATIONS

Optimization	Status	Evidence in repo
Incremental async de-delay	Adopted	<code>llm2decoder_delay_async_chunk</code> stores <code>_DelayAsyncRequestState</code> per request. Each new delay row fills restored-frame slots in a bounded ring buffer, and only completed rows are flushed.
Vectorized sync de-delay	Adopted	<code>_apply_de_delay_pattern</code> builds tensor index grids <code>frame_idx</code> , <code>ch_idx</code> , and <code>src_steps</code> ; it does not loop over codebooks.
Packed audio embeddings	Adopted for Delay	<code>moss_tts_delay_ar_stage.py</code> uses <code>audio_embedding_weight[n_vq, vocab, hidden]</code> . The Local branch still uses separate embedding modules.
Packed/fused audio heads	Adopted for Delay	Delay uses <code>audio_lm_head_weight[n_vq, vocab, hidden]</code> and computes <code>audio_logits</code> with <code>torch.einsum("bh,qvh->bqv", ...)</code> .
Vectorized audio sampling	Adopted for Delay	Delay computes active request/codebook indices with <code>torch.nonzero</code> and samples only <code>audio_logits[active_b, active_q, :]</code> .
Avoid CPU history copy when unused	Adopted for Delay	<code>store_next_audio_row(..., track_history=repetition_penalty > 1.0)</code> skips CPU history updates when the benchmark setting <code>repetition_penalty=1.0</code> is used.
Skip Stage 0 hidden-state CPU copy	Adopted for Delay async	<code>gpu_ar_model_runner.py</code> sets <code>hidden_states_cpu=None</code> for <code>llm2decoder_delay_async_chunk</code> .
Suppress unnecessary scheduler payloads	Adopted for Delay async	<code>OmniARScheduler</code> sets <code>_suppress_pooling_output_to_client</code> for the Delay async processor, while still passing the payload to the chunk-transfer adapter.
Connector state cleanup	Adopted	<code>cleanup_sender</code> removes <code>_moss_tts_delay_async_states</code> , <code>_moss_tts_delay_audio_pad_code</code> , and related per-request caches.
Dynamic async chunking	Adopted	<code>moss_tts_delay_async.yaml</code> sets <code>initial_codec_chunk_frames: 3</code> and <code>codec_chunk_frames: 50</code> .

TABLE II
NATIVE/HUGGINGFACE-INSPIRED TECHNIQUES AND CURRENT STATUS

Technique	Status in this repo	Notes
Packed audio embeddings	Adopted for Delay	Implemented through <code>audio_embedding_weight</code> ; Local still uses module lists.
Packed/fused audio heads	Adopted for Delay	Implemented through packed head weights and <code>einsum</code> .
Tensorized de-delay	Adopted	Used by the sync Delay processor.
Incremental async de-delay	Adopted	Used by the async Delay processor.
Avoid repeated full-buffer de-delay	Adopted	Async Delay updates restored frame slots incrementally instead of rebuilding and de-delaying the full matrix each step.
Avoid CPU history copy when unused	Adopted for Delay	Active when <code>repetition_penalty=1.0</code> .
Batched Stage 1 stateless decode in sync	Preserved	Stage 1 can batch multiple complete code sequences in sync mode.
Larger chunked decode for throughput	Partially adopted	Delay async uses a small first chunk and larger steady-state chunks.
Single-process local tensor loop	Not adopted	vLLM async still separates Stage 0, connector transfer, and Stage 1.
No shared-memory/IPC transfer	Not adopted	Async vLLM-Omni still serializes chunks through the connector.
No per-step scheduler handoff	Not adopted	vLLM still schedules and dispatches autoregressive steps.
GPU tensor state for request FSM	Not adopted	Request state remains Python-controlled.
No side effects inside <code>forward()</code>	Not adopted	Stage 0 mutates per-request generation state during <code>forward()</code> .
CUDA graphs or compile	Not adopted	MOSS configs use <code>enforce_eager: true</code> .
Offline full-sequence codec decode in async	Not adopted	Async mode uses streaming codec chunks by design.
Fully tensorized sampling and state update	Partially adopted	Delay audio sampling is vectorized, but request state updates are still Python-controlled.

E. Evaluation Metrics

For our experiments, we conduct both inference and service benchmarks. We utilize the following metrics:

- 1) RealTime Factor (RTF): $RTF = Wall_T / Audio_T$
measuring the amount of time it takes process audio
- 2) First Chunk Latency (FCL or TTFA):
 $FCL = Time_{firstChunk} - Time_{request}$
amount of time to stream first chunk (Async Modes)
- 3) Throughput: $TP = Audio_T / Wall_T$,
Amount of audio generated per second.

RTF allows us to measure a ratio between the total time to generate audio and the length of an audio clip; this showcases whether the model is able to generate a clip in a shorter or longer time than its length. A smaller RTF such as 0.5, signifies that the model was able to generate a longer audio

clip in a shorter amount of time and vice versa. This smaller RTF would be something that would be desired in a model, as it allows us to process more audio in a quicker amount of time.

FCL or TTFA is a measurement specifically for the Async variants of the vLLM implementations, as it measures how fast a user would hear audio streamed to them in real time. A smaller FCL or TTFA would be superior, as it means the user would hear audio playback close to the time of their request.

We also measure throughput, which is the inverse of RTF, where we are looking to see how much audio is generated per second of runtime. In this case, a bigger throughput would mean more audio is processed per second, which is the desired behavior. Increasing throughput allows the model to handle more requests and audio per second than that of smaller throughputs.

For RTF and first chunk latency (FCL/TTFA), the exper-

iments use 20 examples from the HuggingFace WikiText dataset with varying sentence lengths, between 10 - 60 words. We ensure the same sets of 20 examples are used across the HuggingFace and vLLM variants to ensure a fair and honest comparison. It is to be noted that for the HF variants of the model and the Sync modes, the reported time for FCL is the same as the total latency, as these models do not generate chunks and only stream audio after the whole request is processed.

For throughput, the experiments use static fixed sentences to reduce variance across serving configurations, run three times at each batch size and averaged to reduce the variance. We test batch sizes of 1, 4, 16, 64, 128, and 256 in our experiments to exponentially increase the size of requests. Each of the batch sizes greater than 2 were compared against BS = 1 to see the speedup relative to each model. We note that HuggingFace does not contain continuous batching by default; therefore, we compare it to static batching, which could be considered even more challenging as it does not have any continuous batching overhead like the vLLM implementations.

IV. RESULTS

A. Experiment Setup

1) *Hardware*: For our experiments, we utilized the Vast AI platform and Google colab to run our experiments. In both scenarios we utilized the A100 GPU with 80 GB of VRAM, and had at least 100+ GB of disk space in order to store the model weights and other miscellaneous overhead.

2) *Baselines*: We compare each vLLM-Omni implementation against its corresponding HuggingFace/native MOSS-TTS baseline.

3) *Model Configurations*: Default configurations in the YAML files were used.

4) *Runtime Environment*: CUDA 12.8 with vLLM 0.19.1 must be utilized and were the main points that hindered runtime experience.

B. Throughput

TABLE III
THROUGHPUT COMPARISON: MOSS-TTS-LOCAL

BS	HF		ASYNC		SYNC	
	Tput	Spd	Tput	Spd	Tput	Spd
1	0.299	1.00×	0.536	1.00×	0.528	1.00×
4	0.973	3.25×	0.705	1.32×	1.664	3.09×
16	3.402	11.38×	1.944	3.63×	4.002	7.57×
64	10.139	33.91×	7.470	14.91×	13.574	25.69×
128	11.610	38.83×	11.866	22.15×	21.290	40.29×
256	15.148	50.66×	18.265	34.09×	26.718	50.56×

In tables III and IV, we compare the local and delay vLLM variants against their respective HF baselines. In general, we can see that as the batch size continues to increase, the vLLM variants tend to have superior performance across both model variants. This is especially clear in the Local variants of the

TABLE IV
THROUGHPUT COMPARISON: MOSS-TTS-DELAY

BS	HF		ASYNC		SYNC	
	Tput	Spd	Tput	Spd	Tput	Spd
1	0.713	1.00×	1.664	1.00×	1.566	1.00×
4	2.877	4.04×	3.590	2.16×	3.839	2.45×
16	7.088	9.94×	13.145	7.90×	14.152	9.04×
64	18.612	26.10×	19.055	11.45×	40.291	25.73×
128	21.667	30.39×	20.204	12.14×	54.463	34.78×
256	25.006	35.07×	21.817	13.11×	69.956	44.68×

model, with the Local Sync model outputting 26.718, almost a 1.8 times increase in throughput over the vanilla HF model. The Async version also shows improvements, at 18.265.

A similar trend can be seen in the Delay variants as well, with the Sync model outputting 69.956, a 2.8 times increase over HF's 25.006 for the 256 batch size. In this case, we can see Async mode actually falter off when it hits 128 and 256 batch sizes, but this is expected behavior, as we actually run out of memory at these batch sizes and it leads to the model defaulting to single requests to accommodate for memory cost. This is consistent with the additional delay-pattern buffering and chunked decoder invocation required by streaming. Had we run the two stage architecture on a multi GPU setup, more than likely Async would have exceeded the HF baseline at both 128 and 256.

Overall, the throughput benchmark showcases the strength of vLLM processes, and in the throughput benchmark the sync variants of the model truly shine, showcasing an exponential increase in throughput.

C. RTF and FCL (or TTFA)

Table V highlights the main benefit of async chunking: reducing TTFA by allowing the decoder to emit audio before the full code sequence has been generated. For MOSS-TTS-Local, the first chunk latency drops from 21.860 s in the HuggingFace native baseline to 0.557 s with async chunking. For MOSS-TTS-Delay, first chunk latency drops from 5.002 s in the HuggingFace native baseline to 0.970 s with async chunking. The Local async path emits many more chunks on average because it can flush complete frame-major rows without waiting for a delay-pattern ramp-up.

We also see the decrease in RTF across both async and sync variants in comparison to HF models. This is not only consistent with our throughput benchmarks, but it also showcases how much more efficiently the vLLM models can handle processing of requests in comparison to HF Native models.

V. DISCUSSION

The integration confirms that MOSS-TTS can fit vLLM-Omni's staged serving abstraction, but only with model-specific adapters. The important engineering point is that Stage 0 and Stage 1 communicate through a stable code interface: flat 32-codebook RVQ tokens. Once both MOSS variants are

TABLE V
TTS PROFILING SUMMARY ACROSS CONFIGURATIONS

Metric	Local Model		Delay Model		HF Native	
	Async	Sync	Async	Sync	Delay	Local
RTF (mean)	1.876	1.202	0.470	0.489	1.730	3.292
Total Latency (s, mean)	11.859	10.931	2.901	2.958	5.002	21.860
First Chunk Latency (s)	0.557	10.931	0.970	2.948	5.002	21.860
Num Chunks (mean)	27.00	1.00	3.25	1.00	1.00	1.00

normalized into that interface, they can share the same CAT decoder implementation.

The Local variant is simpler for streaming because each Stage 0 decode step produces one complete frame-major code row. This makes the first audio chunk available after only a small number of generated frames. The Delay variant has a stronger autoregressive backbone, but its delay pattern creates an inherent streaming ramp-up: the integration must wait until enough shifted quantizer positions have arrived to reconstruct complete rows. The async Delay processor therefore optimizes around a small first flush and larger later flushes.

The throughput tables also show a tradeoff between sync and async operation. Sync mode generally achieves the best high-batch throughput because it decodes a larger complete code sequence with less streaming overhead. Async mode is designed for TTFA and interactive serving rather than peak batch throughput. We attribute the throughput gains primarily to vLLM-Omni’s stage-aware batching and scheduling, along with PagedAttention and fused-kernel support inherited from vLLM’s Qwen3 implementation.

Despite the throughput and latency improvements, the current integration has three main limitations. First, the MOSS-TTS-Local local transformer is still not fully implemented as a native vLLM-compatible transformer component, so it cannot fully benefit from PagedAttention and fused kernels in the same way as the Qwen3 global backbone. Second, CUDA Graph support remains incomplete for multiple batch sizes because of incompatibility between `torch.compile` and the current `OmniOutput` path. Third, profiling is difficult to compare directly between HuggingFace and vLLM-Omni because the integrated models have different execution scopes and architecture boundaries.

We also wish to further develop CUDA Graph support, multi-GPU deployment that places Stage 0 and Stage 1 on separate GPUs, additional prompt styles such as reference audio and ambience/noise-conditioned prompts, and more realistic real-time serving benchmarks with random request arrivals and more complex text examples.

VI. CONCLUSION

This project integrates MOSS-TTS-Local and MOSS-TTS-Delay into vLLM-Omni as two-stage TTS pipelines. The work adds custom Stage 0 implementations for the Local and Delay autoregressive generators, a shared CAT codec decoder stage, sync and async inter-stage processors, and YAML configurations for batch and streaming execution. The current results

show that vLLM-Omni can improve high-batch throughput over HuggingFace baselines and that async chunking can substantially reduce time to first audio, especially for the Local model.

REFERENCES

- [1] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [2] Y. Yin et al., “vLLM-Omni: Fully disaggregated serving for any-to-any multimodal models,” arXiv:2602.02204, 2026.
- [3] OpenMOSS Team, “MOSS-TTS Family,” GitHub: <https://github.com/OpenMOSS/MOSS-TTS>, released February 10, 2026.
- [4] OpenMOSS Team, “MOSS-Audio-Tokenizer: Scaling audio tokenizers for future audio foundation models,” arXiv:2602.10934, 2026.
- [5] Qwen Team, “Qwen3-TTS technical report,” arXiv:2601.15621, 2026.

APPENDIX A

AI DISCLOSURE STATEMENT

We utilized AI in the following manner: Claude, ChatGPT, and GitHub Copilot were used to understand the two-stage pipeline of vLLM-Omni, debugging compatibility issues between vLLM-Omni and MOSS-TTS. It was also used to understand the codebase in general, especially for how profiling was implemented and the internal workings of vLLM/vLLM-Omni. We also utilized AI such as copilot’s autofill function for loops, boilerplate code, and cleaning up prose. Also, debugging environment setup for running models on Vast AI.

APPENDIX B

GITHUB REPOSITORY

<https://github.com/candylyt/vllm-omni>

APPENDIX C

WANDB TRACKING DASHBOARD

<https://wandb.ai/ac5905-columbia-university/hpml-final-project/runs/xf9dt56?nw=nwuserac5905>